VerbalVis: Hands-Free Exploratory Data Analysis with Full-Duplex Speech Interaction

Anonymous Author(s)

***Abstract*—Exploratory data analysis (EDA) is fundamentally a process of *analytical intent evolution*: analysts continuously revise their direction as observations surface hypotheses, refute them, and prompt further questions. Existing speech- and language-driven visual analytics (VA) systems are largely turn-based, confining intent revision to discrete handoff points between user and system. Recent full-duplex speech-to-speech models make a different interaction regime possible: one in which the analyst can redirect an analysis mid-utterance while a response is still unfolding. We treat barge-in as a *structural supersession signal*: the interruption event indicates that the current response and its pending actions may no longer be valid, while the redirecting utterance supplies the semantic specification of the revised intent. We present VerbalVis, a hands-free, full-duplex conversational VA system built on `gpt-realtime-2` and an in-memory DuckDB analytics layer. VerbalVis implements a reusable supersession control protocol that cancels or truncates stale response work, invalidates in-flight tool calls through response epochs, captures the redirecting utterance, and replans against a dashboard state that reflects prior valid tool calls. We contribute (1) an *Intent Evolution* framework identifying three recurring revision types observed in conversational EDA — *Goal Shift*, *Hypothesis Correction*, and *Scope Narrowing*; (2) a 15-operation tool taxonomy aligned to the Brehmer–Munzner *why* and Yi et al. *how* axes, with a representative dashboard-operation subset realized in the prototype; (3) the VerbalVis prototype on the Olist Brazilian e-commerce dataset; and (4) an evaluation plan comprising a case study, an $N{=}12$–16 within-subjects study whose primary comparison is VerbalVis-FullDuplex versus VerbalVis-NoBarge, a supplementary cascaded turn-based baseline, and a 250-trial technical benchmark of the supersession mechanism.**

***Index Terms*—Visual analytics, conversational interfaces, natural language interfaces, speech interaction, large language models, full-duplex dialogue, exploratory data analysis.**

I. Introduction

EXPLORATORY data analysis (EDA) is an inherently iterative process. An analyst glances at a trend, forms a hypothesis, queries the data for confirmation; the data refutes the hypothesis; the analyst pivots to a related question; that question splinters into sub-questions; and so on [1], [2]. The *shape* of this trajectory — where the analyst turns, why, and how quickly — is not incidental; it is the substance of sense-making. Supporting EDA in a conversational visual analytics (VA) system therefore means supporting not query answering alone but *analytical intent evolution*: the continuous, often mid-utterance revision of analytical direction.

The structural limitation of current conversational VA. Representative natural-language interfaces for visualization — DataTone [3], Eviza [4], NL4DV [5], and recent LLM-based systems including Data Formulator 2 [6], LightVA [7], and DashChat [8] — are organized around turns: the user speaks or types, the system processes, the system responds, and the user speaks again. Intent revision is consequently confined to discrete handoff points. When the system is mid-response with an analysis that has already deviated from the user's interest, the user has no recourse except to wait, then issue a correction, then wait again. The cost compounds in voice modalities, where responses unfold at speaking rate and a misdirected explanation may consume tens of seconds before the user regains the floor.

The key observation. Full-duplex speech-to-speech models such as OpenAI's `gpt-realtime-2` expose a different temporal affordance: the user can interrupt the system at any instant, and the system can detect the interruption, cancel or truncate the in-flight response, and continue from the newest user input [9]. Crucially for VA, this does not mean that the interruption event by itself determines the user's analytical meaning. A barge-in may be a backchannel, an ASR correction, accidental noise, a device check, or a true analytical redirect. We therefore treat barge-in more narrowly and more operationally: it provides a *structural supersession signal* that the current response and its pending actions may no longer be valid, while the redirecting utterance provides the semantic specification of the revised intent. This reframing requires an architecture that first prevents stale speech and tool work from committing, then interprets the newest user utterance against the preserved dashboard state.

VerbalVis. We present **VerbalVis**, a hands-free, full-duplex conversational VA system. The contribution of VerbalVis is not a new model-level mechanism for barge-in detection; it is a reusable *supersession control protocol* for connecting interruption-aware speech with stateful analytical tools. The protocol combines (i) native speech-onset and response-cancellation primitives from `gpt-realtime-2`; (ii) response epochs that assign ownership to tool calls and discard late results from superseded turns; (iii) schema-based analytical tools over an in-memory DuckDB backend; and (iv) continuously reinjected dashboard context that grounds post-interruption replanning in what the analyst actually sees. The current prototype instantiates this protocol on the Olist Brazilian e-commerce dataset with a Vue/Vega-Lite dashboard and tool operations for filtering data, removing filters, highlighting views, appending charts, and deleting views. After every successful tool call, the backend re-queries affected views, sends a `views_update` event to the frontend, and reinjects a compact dashboard snapshot — active filters,

Manuscript submitted to IEEE Transactions on Visualization and Computer Graphics. Source code, prompts, and study materials available at the project repository upon publication.

highlighted view, available views, filtered row count, and per-view statistics — as system context. Prior LLM-agent VA systems achieve subsets of this combination: LightVA [7] provides agentic task execution but relies on runtime code generation, while DashChat [8] provides conversational LLM coordination for dashboard authoring rather than analytical use. VerbalVis focuses on the control problem that arises when spoken redirection and analytical tool execution overlap in time.

Contributions. We make four contributions:

1) An **Intent Evolution framework** that conceptualizes intent revision in EDA and identifies three recurring revision types observed in conversational analytical sessions — *Goal Shift*, *Hypothesis Correction*, and *Scope Narrowing* — with their triggering conditions and dashboard-level consequences. These types are conceptual categories used to organize observation; VerbalVis treats them with uniform downstream processing rather than per-type runtime dispatch.
2) A reusable **supersession control protocol** that connects native barge-in handling, response epochs, stale-tool invalidation, schema-based tool calling, and continuously reinjected dashboard context, enabling intent revision to move from discrete handoff points to a continuously available action without introducing new model-level mechanisms.
3) The **VerbalVis system**: a prototype implementation with a 15-operation analytical tool taxonomy, a representative dashboard-operation subset realized end-to-end, realtime speech I/O, Vega-Lite dashboard views, and logging for response latency, tool latency, stale calls, and dashboard state.
4) A **case study and evaluation plan**: an N=12–16 within-subjects study whose primary causal comparison is **VerbalVis-FullDuplex** versus **VerbalVis-NoBarge**, a supplementary **VerbalVis-Cascade** ecological baseline, and a 250-trial technical benchmark that isolates mechanism-level effects of the supersession path.

The remainder of the paper is organized as follows. Section II positions our work relative to NLI-for-visualization, LLM-agent VA, sensemaking theory, and full-duplex conversational AI. Section III introduces the Intent Evolution framework. Section IV describes the design and implementation of VerbalVis. Section V reports a case study illustrating all three revision types, Section VI presents the user-study and benchmark design, and Sections VII–VIII discuss limitations and conclude.

II. Related Work

A. Natural-Language Interaction for Visualization

Natural-language interfaces for visualization (V-NLIs) investigate how users express analytical operations in language and how systems ground those expressions in data, visualization specifications, and the current visual state. Early systems established that this problem is not simply text-to-chart generation. DataTone exposes alternative interpretations through interactive ambiguity widgets rather than silently committing to a single parse [3], whereas Eviza grounds a probabilistic grammar in the active dataset and visualization to support filtering, comparison, selection, and navigation of an existing view [4]. NL4DV subsequently externalized an interpretation as a structured analytic specification containing inferred attributes, analytical tasks, ambiguities, and candidate Vega-Lite specifications [5]. Together, these systems framed language interaction as coordination among linguistic intent, data semantics, analytical tasks, and visual state.

Later work broadened V-NLIs from isolated commands toward contextual and multimodal analysis. Orko combines natural language with direct manipulation for visual network exploration, allowing speech and graphical selection to complement one another [10]. The conversational extension of NL4DV supports follow-up utterances that augment, remove, or replace components of earlier specifications while maintaining multiple dialogue contexts [11]. Snowy recommends executable follow-up utterances based on the current data and interaction context, thereby supporting both analytical guidance and discoverability of the language interface [12]. These systems show that prior V-NLIs can be iterative, context-aware, conversational, and multimodal; speech is not necessarily used only as a substitute for typed text.

The remaining distinction is temporal. In most V-NLIs, contextual refinement is processed after a user utterance or system update has reached a recognizable boundary. Even systems that continuously observe conversation do not necessarily allow a user to speak over an incrementally generated system response, terminate response work associated with a superseded request, and prevent a late analytical result from overwriting the revised state. VerbalVis builds on contextual and multimodal V-NLIs but shifts revision from a between-turn operation to a within-response operation. Its contribution in this research stream is therefore not a new language-to-visualization parser, but the coupling of full-duplex speech, schema-grounded analytical tools, and dashboard-state grounding so that an interruption can redirect the active analysis while the assistant is still speaking.

B. LLM-Powered and Agentic Visual Analytics

Large language models extend visualization interfaces beyond semantic parsing by generating data transformations and visualizations, planning analyses, executing code or tools, and organizing analytical results. LIDA, for example, decomposes visualization generation into data summarization, goal generation, code generation, execution, and refinement [13]. Data Formulator 2 blends graphical specification with natural-language prompting and delegates required data transformations to AI; its data-thread history supports revisiting completed states, reusing designs, and branching into alternative authoring directions [6]. These systems establish iterative refinement and non-linear analytical histories as prior capabilities.

Agentic systems expand the unit of work from a requested chart to a multi-step analytical process. LightVA uses a planner–executor–controller architecture to recursively decompose high-level goals into data-analysis and visualization

tasks [7]. ProactiveVA grounds an LLM-based UI agent in interaction logs and the active interface, allowing the agent to perceive visual-analytics state and provide context-sensitive assistance [14]. WaitGPT makes streamed analytical code visible as editable data operations, enabling users to inspect, modify, rerun, and resume portions of an LLM-generated analysis before the full process has completed [15]. DashChat addresses another stage of the visualization lifecycle: it coordinates LLM-powered agents and design patterns to conversationally generate and modify industrial dashboard prototypes [8]. Whereas DashChat supports dashboard authoring from textual requirements, VerbalVis operates on a live dashboard during exploratory analysis.

The closest prior systems thus expose complementary forms of human control. Data Formulator 2 supports revision across persistent authoring states; LightVA exposes task planning and execution; WaitGPT supports intervention in an ongoing code-based process; and ProactiveVA acts on a live interface. However, these controls are primarily expressed through text, direct manipulation, completed-state branching, or explicit inspection of intermediate operations. They do not report a unified coordination mechanism in which a spoken intervention during agent output simultaneously terminates stale speech, changes the ownership or validity of pending analytical actions, and resumes from the last valid dashboard state.

VerbalVis addresses this temporal coordination problem rather than claiming novelty from LLM agency alone. Its schema-based tools constrain analytical actions to auditable dashboard operations, while each response and tool task is associated with the current interaction epoch. When a new utterance supersedes an active response, VerbalVis cancels in-flight work where possible and rejects late results from stale epochs before they can update the dashboard. Dashboard context is then reinjected for subsequent planning. LightVA and DashChat remain important conceptual precedents for agent-based and conversational VA, and their turn-mediated interaction regime motivates the cascaded baseline used as supplementary ecological context in our evaluation; VerbalVis differs by studying how human control is exercised during response production rather than only between completed turns.

C. Full-Duplex Conversational AI and Interruption

Full-duplex conversational AI studies interaction in which listening and speaking proceed concurrently and newly arriving user speech can alter an agent's ongoing behavior. This property is distinct from streaming, push-to-talk, and ordinary multi-turn dialogue. A streaming system may incrementally recognize or synthesize speech while still enforcing alternating turns; a multi-turn system may preserve conversational context without listening during its own output. Full-duplex interaction additionally requires policies for overlap, backchannels, user barge-in, accidental noise, response termination, and the effect of an interruption on an underlying task.

Duplex Conversation operationalized full-duplex interaction through user-state detection, backchannel selection, and barge-in detection in a deployed spoken-dialogue system [16]. Wang et al. proposed an LLM-controlled dialogue scheme in which perception and speech generation run concurrently and control tokens determine whether the system should respond, wait, or interrupt [17]. SyncLLM introduces explicit temporal modeling so that an autoregressive language model can generate dialogue synchronously with a real-world clock, supporting overlap, rapid turn-taking, and backchannel behavior [18]. Moshi instead models user and assistant audio in parallel streams, avoiding a strict segmentation into alternating speaker turns and supporting real-time speech-to-speech interaction [19]. These works primarily advance conversational timing and architectures for simultaneous listening and speaking.

Recent research has begun to connect full-duplex dialogue with structured action generation. DuplexSLA jointly decodes assistant audio and a textual action stream on a shared temporal grid, allowing planning text, control decisions, and structured tool calls to be produced while speech continues [20]. Full-Duplex-Bench-v3 evaluates real voice agents using human speech containing hesitation and self-correction together with scenarios that require chained API calls, and reports accuracy, latency, and turn-taking behavior across full-duplex and cascaded systems [21]. These developments establish in-conversation tool use as an emerging problem; VerbalVis is not the first full-duplex system to call tools.

The unresolved issue for visual analytics is action validity within a persistent, mutable visual state. Generic tool-use benchmarks evaluate whether the final sequence and arguments of API calls are correct, but a dashboard may already contain filters, highlights, derived views, and results produced by previous calls. If a user revises intent while a response is speaking, an old tool task may already be queued or executing, and its late result may arrive after the revised analysis has begun. Stopping audio alone does not preserve state consistency. VerbalVis therefore extends full-duplex interruption from conversational floor control to cross-channel coordination: it terminates or truncates stale speech, invalidates tool tasks associated with superseded response epochs, rejects late stale results, and replans against the current dashboard snapshot. The contribution lies in coordinating these mechanisms for visual analysis, not in introducing a new speech model or interruption detector.

D. Sensemaking and Analytical Intent Evolution

Sensemaking research explains why analytical direction cannot be assumed to remain fixed. Russell et al. describe sensemaking as a search for representations that reduce the cost of organizing and interpreting information; when a representation becomes inadequate, analysts must change how evidence is structured [22]. Pirolli and Card characterize analysis as movement between information-foraging and sensemaking loops, through which evidence and hypotheses repeatedly reshape one another [1]. Studies of enterprise analysis likewise show that analysts work with incomplete goals, repeatedly reformulate questions, and alternate among data acquisition, transformation, exploration, model construction, and communication [2]. Analytical revision is therefore a normal part of exploratory work rather than an exceptional recovery from user error.

Visual analytics models and systems make aspects of this process explicit. The Knowledge Generation Model connects exploration, verification, and knowledge-generation loops, emphasizing reciprocal transitions among data, visualizations, hypotheses, insights, and knowledge [23]. Analytical-provenance research distinguishes interaction, insight, and rationale provenance [24]. SensePath combines interaction logs with think-aloud data to reconstruct sensemaking trajectories [25], while Graphical Histories supports revisitation, comparison, and branching over prior visualization states [26]. Longitudinal evidence further shows that analysts change their organizational and sensemaking strategies as familiarity, evidence, and task understanding develop [27]. Collectively, this literature provides theoretical and empirical grounding for studying evolving analytical goals, hypotheses, and scopes.

VerbalVis uses *Goal Shift*, *Hypothesis Correction*, and *Scope Narrowing* as a design-oriented coding lens for this evolution. Goal Shift denotes a change in the primary analytical objective or frame. Hypothesis Correction revises or rejects an interpretation while retaining continuity with the broader problem. Scope Narrowing preserves the high-level objective while restricting the relevant population, period, region, variables, comparison set, or level of detail. These categories synthesize related ideas from reframing, hypothesis verification, conceptual restructuring, focused search, and changes of analytical strategy. They are paper-level analytical categories rather than runtime classifiers, mutually exclusive tool-dispatch branches, or an exhaustive taxonomy of sensemaking behavior.

The difference from provenance and branching systems is again temporal and operational. Provenance usually records a trajectory after actions occur, and a branch usually begins from a stable or completed state. Such systems help analysts understand, revisit, or compare the path of analysis, but they do not generally specify what should happen when a revised intention is voiced while an AI-generated explanation and its associated actions are still active. VerbalVis connects the cognitive account of evolving intent to a full-duplex control mechanism: a spoken intervention can supersede active response work, prevent stale analytical results from changing the view, and resume the conversation from dashboard-grounded context. Thus, sensemaking theory explains *why* intent evolves, while VerbalVis investigates how a conversational VA architecture can respond when that evolution is expressed.

Taken together, these four research streams establish complementary foundations. V-NLIs ground language in data and visual state; agentic VA systems plan and execute multi-step analytical work; full-duplex models permit intervention during speech and increasingly support tool use; and sensemaking research explains why analytical objectives and hypotheses evolve. VerbalVis addresses the coordination problem at their intersection: maintaining a consistent analytical state when a spoken intervention supersedes an active response and the visualization actions associated with it.

III. The Intent Evolution Framework

A. A Cyclic Model of Intent Revision

We model EDA as a cycle whose nominal trajectory is repeatedly perturbed by revisions:

Observation → Suspicion → Interruption → Intent Revision →

Analytical Replanning → New Visualization → Observation

The cycle is conventional in sensemaking literature [1]; our contribution is to identify *Interruption* as an explicit, machine-detectable supersession cue that, in a full-duplex system, can be acted on synchronously rather than discovered after the fact from a transcript.

Scope of the framework. The three revision types below are conceptual categories for describing observed analytical behavior, not separate runtime mechanisms. VerbalVis does not classify barge-in events into these types during execution. Instead, it handles all interruptions through the same downstream process: cancel or truncate stale response work, invalidate stale tool calls, accept the redirecting utterance, and regenerate against the latest dashboard context. Type-aware response policies are an important future extension, but they are not required for the architectural contribution in this paper.

B. Three Revision Types

Across our case study sessions and pilot interactions, we observe that intent revisions cluster into three recurring types, each with characteristic triggering conditions:

Type 1: Goal Shift. The analyst's stated goal pivots to a different analytical question. Trigger: the current direction is mismatched with the analyst's actual interest, often because the system has chosen a plausible-but-tangential expansion. Example: while the system narrates category-level sales decomposition for November 2017, the analyst interrupts with "I care about ratings, not sales."

Type 2: Hypothesis Correction. The analyst rejects the explanatory hypothesis under construction and substitutes an alternative. Trigger: a conflict between the system's tentative explanation and the analyst's domain prior. Example: the system attributes low-rated electronics to "product quality issues"; the analyst interrupts with "I think it is logistics, not the goods."

Type 3: Scope Narrowing. The analyst voluntarily contracts the analytical scope to a subset they already know to be relevant. Trigger: the analyst possesses business knowledge that renders the full sweep unnecessary. Example: during a national breakdown by state, the analyst interrupts with "Only São Paulo."

These categories are not exhaustive, but in our case study they cover every revision we observed. Section V illustrates each in detail.

C. Why Full-Duplex Matters

In a turn-based system, intent revisions can only occur at handoff points; a misdirected response must run to completion before the revision is even expressible. In a full-duplex system,

[Figure placeholder] System architecture: Frontend (Vue/Vega-Lite) ↔ Backend (FastAPI relay + DuckDB) ↔ OpenAI Realtime (`gpt-realtime-2`). Realtime Layer (audio streaming, VAD, barge-in, response cancel) on the left; Analytics Layer (tool calls, DuckDB, dashboard update) on the right; shared dashboard state in the middle.

Fig. 1. VerbalVis architecture. The Realtime Layer handles conversational coordination; the Analytics Layer executes tool calls asynchronously and updates the dashboard state, which is reinjected as system context after every tool call.

intent revisions are continuously available: the user reclaims the floor by speaking, and the system, on detecting speech onset, can cancel the in-flight response and re-plan. The framework thus motivates a direct system-level requirement — the controller must support synchronous cancellation, state preservation across the cancellation, and rapid replanning against the unchanged dashboard — which we address in Section IV.

IV. System Design and Implementation

A. Design Goals

The framework yields three design goals:

- **DG1: Continuous intent revision.** Support revision at any moment, not only at turn boundaries.
- **DG2: Rapid replanning after supersession.** Once an interruption is detected during assistant output, the system must prevent stale response work from committing, interpret the redirecting utterance, and re-plan against current dashboard state with minimal added latency.
- **DG3: Dashboard–intent synchrony.** The shared dashboard must always reflect the latest applied operations and must serve as the canonical source of truth for the LLM, so that explanations are grounded in what the user sees rather than in the model's prior outputs.

B. Architecture Overview

VerbalVis comprises a Vue 3 + Vega-Lite frontend, a FastAPI backend with an in-memory DuckDB analytics layer, and a relay that bridges a browser WebSocket to the OpenAI Realtime WebSocket (Fig. 1). The backend exposes a single `/ws` endpoint; on connection, it constructs a `RealtimeSession` that runs two concurrent async loops — one forwarding client audio frames to OpenAI, one demultiplexing OpenAI events to the client and dispatching tool calls.

Realtime layer. `gpt-realtime-2` streams 24 kHz PCM audio in both directions. Depending on the experimental condition, the frontend uses local voice gating or push-to-talk, while the Realtime session may also expose server-side speech events such as `input_audio_buffer.speech_started` and `...speech_stopped`. On interruption-relevant events, the backend increments a *turn epoch*, records the in-flight `response_id` as invalidated, and marks tool tasks from the cancelled response as stale. In push-to-talk mode the backend additionally sends `response.cancel`; in server-interruption mode, it relies on the Realtime session's native interruption behavior and applies the same stale-result discipline locally.

Analytics layer. Tool calls (`response.function_call_arguments.done`) are dispatched to handlers backed by DuckDB queries. DuckDB holds two derived fact tables built from seven Olist CSVs at startup: `fact_order` (one row per delivered order, carrying order-grain attributes including `order_month`, `review_score`, `customer_state`, `delivery_days`, and per-order payment) and `fact_item` (one row per order item, additionally carrying `product_category` and per-item revenue price + freight). The two-table design lets revenue retain the correct semantics under both order-grain and item-grain aggregations; cross-grain filters on `product_category` from the order table are rewritten as `order_id IN (SELECT ...FROM fact_item WHERE ...)` subqueries by a shared `build_where` helper.

VerbalVis employs real-time full-duplex audio interaction for conversational coordination, while analytical operations are executed asynchronously through tool calls that update the dashboard state.

C. Tool Taxonomy

Following the Brehmer–Munzner multi-level typology [28] and Yi et al.'s how-axis [29], we organize VerbalVis operations along three dimensions: *why* (analytical goal), *how* (interaction primitive), and *what* (target of the operation: data, view, layout, insight). Table I lists a 15-operation taxonomy for the design space. The current prototype realizes five dashboard operations end-to-end as schema-based tools: `filter_data`, `remove_filter`, `highlight_visual`, `append_visual`, and `delete_visual`. This subset is deliberately narrow: it covers the operations needed for the running example and user-study tasks while keeping tool selection predictable under realtime interruption.

The implemented tools are deliberately minimal. `filter_data` centralizes global dataset narrowing (with a `field=null` convention for clearing, an `append` flag for AND-accumulation, and operator support for `eq`, `neq`, `in`, `gte`, `lte`, `between`); `remove_filter` removes one field constraint without disturbing the rest; `highlight_visual` redirects attention without changing data; `append_visual` creates a new chart at item or order grain based on whether `product_category` appears in the encoding; and `delete_visual` removes views from the shared workspace. This minimality is intentional: most analytical state changes in our scenarios can be expressed by composing these operations, which keeps the model's tool-selection decision tree shallow and reduces the surface area on which the LLM can hallucinate.

D. Layered Prompt Design

The system prompt is constructed in four labeled sections, in line with the prompt-structure recommendations of the GPT-realtime-2 prompting guide [30]. *Identity* fixes the assistant

TABLE I
VERBALVIS TOOL TAXONOMY. FIVE OPERATIONS ARE IMPLEMENTED END-TO-END AS SCHEMA-BASED TOOLS (MARKED ✓); THE REMAINDER DEFINE THE BROADER DESIGN SPACE. *Why* FOLLOWS BREHMER–MUNZNER; *How* FOLLOWS YI ET AL.; *What* IDENTIFIES THE OPERATION TARGET.

Tool	Why	Why-Detail	How	What	Impl.
`filter_data`	Discover	Locate	Filter	Data	✓
`remove_filter`	Discover	Locate	Filter	Data	✓
`aggregate_data`	Discover	Summarize	Abstract/Elaborate	Data	
`compare_data`	Discover	Compare	Connect	Data	
`change_chart_type`	Discover	Explore	Encode	View	
`change_encoding`	Present	—	Encode	View	
`change_sort`	Discover	Compare	Reconfigure	View	
`highlight_visual`	Discover	Locate	Select	View	✓
`focus_visual`	Discover	Explore	Select+Reconfigure	View	
`append_visual`	Discover	Explore	Encode	View	✓
`move_visual`	Present	—	Reconfigure	Layout	
`delete_visual`	Discover	Identify	Reconfigure	Layout	✓
`replace_visual`	Discover	Explore	Reconfigure	Layout	
`explain_visual`	Discover	Identify	Connect	Insight	
`summarize_dashboard`	Discover	Summarize	Abstract/Elaborate	Insight	

as a collaborative data analyst, instructs language mirroring, and prescribes the opening behavior (greet, introduce the Olist dataset, walk through the four base views, then invite the user to choose a direction — explicitly without preemptively highlighting any view). *Dashboard Knowledge* provides a bidirectional view-id ↔ Chinese-and-English-label dictionary, the exhaustive field list (forbidding fabricated fields such as "return_rate" or "profit"), and value semantics (e.g., `review_score` 1–2 negative; `customer_state` two-letter codes; revenue in BRL pronounced "reais"). *Tool Usage Rules* specify operator semantics, the cross-grain filter convention, and tool-selection guidelines that explicitly forbid defaulting to the review view for generic prompts such as "what is interesting in the data." *Realtime Rules* legitimize interruption as a high-priority signal and instruct the model to abandon prior explanations and re-evaluate against the latest user utterance.

E. Dashboard State and Context Reinjection

The four base views (`view-trend`, `view-review`, `view-map`, `view-category`) and any user-added `workspace-N` charts are maintained as a single backend list. After every tool call, the backend (i) re-queries all views against the current filter set, (ii) recomputes per-view summary statistics (e.g., peak month, low-score ratio, top state, top category), (iii) pushes a `views_update` event to the frontend for re-render, and (iv) injects a compact textual snapshot of the dashboard — highlighted view, active filters, total filtered row count, and per-view summary — back into the LLM as a `system` message. This *context reinjection* pattern follows the guidance in the GPT-realtime-2 long-context section [30]: the model is never asked to reconstruct dashboard state from prior turns, only to act on what it is given.

F. Supersession Control Protocol

VerbalVis treats barge-in as a control event, not as a semantic classifier. The reusable protocol has five stages. First, the runtime detects possible supersession through either a server speech-start event or a client push-to-talk start during assistant output. Second, the backend increments `turn_epoch`, so newly arriving user input and later tool calls belong to a new response epoch. Third, the system attempts to stop obsolete output through `response.cancel` or `conversation.item.truncate`, depending on the input mode and Realtime event sequence. Fourth, tool calls associated with the superseded response are cancelled when possible and otherwise checked with `is_stale_tool_call` before any result is returned to the model or applied to the dashboard. Fifth, VerbalVis reinjects a compact dashboard snapshot before the next `response.create`, so the revised plan is grounded in the latest valid visual state.

When barge-in is enabled, an interruption is represented by either a server speech-start event or a client push-to-talk start during assistant output. The backend then:

1) Increment the per-session `turn_epoch`; record the in-flight `response_id` as invalidated.
2) Cancel all in-flight tool tasks bound to the cancelled response.
3) Send `response.cancel` or `conversation.item.truncate` when the local input mode requires explicit cancellation or audio truncation.
4) Forward `speech_started` to the frontend so the UI can stop playback immediately.

A double-lock discipline (`openai_send_lock` for outbound messages, `tool_state_lock` for state mutation) and a per-response pending-tool counter ensure that (i) tool results from cancelled responses are discarded by an `is_stale_tool_call` check before being forwarded to the model, and (ii) when a single model turn issues multiple tool calls, only the final completing call triggers a fresh `response.create`, preventing duplicate or racing responses.

No-barge ablation. We define **VerbalVis-NoBarge** as the controlled ablation of the protocol. It uses the same

`gpt-realtime-2` model, voice, prompt, tool schema, dashboard, backend, VAD settings, and context-reinjection logic as VerbalVis-FullDuplex. The only intended difference is policy: user speech during assistant output is recorded or queued, but it does not trigger `response.cancel`, `conversation.item.truncate`, epoch invalidation, or stale-tool discard until the current response completes. This condition separates the effect of the realtime model family from the specific effect of the supersession path.

V. Case Study: Three Forms of Analytical Intent Revision

We illustrate the framework with a single session on the Olist Brazilian e-commerce dataset (96,478 delivered orders; 110,196 order items spanning 73 product categories and 27 states; September 2016 – August 2018). The analyst, a domain non-expert, was asked to explore customer satisfaction with no predetermined goal.

A. Initial State

The system greets the user, names the dataset, and walks through the four base views: monthly order trend, review-score distribution, orders by state, top-15 categories by revenue. The system then asks the analyst what they would like to explore. The analyst responds, "Let's look at the trend."

B. Type 1 — Goal Shift

System (mid-utterance): "Order volume peaks in November 2017, roughly 4× the average month. Let me decompose that by product category to see whether it's a Black Friday effect across categories or concentrated in — " *Analyst (barge-in):* "I care more about ratings, not sales."

The barge-in fires `response.cancel`; the redirecting utterance is interpreted; the controller emits `highlight_visual(view-review)` and the assistant continues: "The review distribution is heavily concentrated at 5 stars but with a non-trivial 1- and 2-star tail. Want me to filter to low-rated orders and see what they share?" The intent has shifted from a sales decomposition to a satisfaction investigation; the analyst was spared the remainder of the category breakdown.

C. Type 2 — Hypothesis Correction

System (mid-utterance): "Electronics-adjacent categories such as `computers_accessories` and `electronics` show below-average ratings. This is often attributable to product quality issues with — " *Analyst (barge-in):* "It's logistics, not the products."

The system invokes `append_visual(scatter, delivery_days, review_score)` followed by `filter_data(review_score, lte, 2)`; the resulting scatter shows the low-rated tail clustering at high delivery days. The assistant continues: "You're right — low-rated orders disproportionately have delivery times above 20 days; the product-quality story doesn't hold up." The intent revision substituted a delivery hypothesis for a product-quality hypothesis, and the assistant aborted its prior explanation rather than completing it.

D. Type 3 — Scope Narrowing

System (mid-utterance): "Among states with the worst delivery times, the northeast — Bahia and Ceará — have averages above 25 days, while — " *Analyst (barge-in):* "Just São Paulo." `filter_data(customer_state, eq, "SP")` is issued; all four base views recompute against the SP-only subset; the assistant pivots to "Within SP, average delivery is 8.4 days; the low-rated tail is much smaller than the national figure." Scope contracted from a national survey to a state-level focus, exploiting the analyst's prior interest in SP without the analyst having had to articulate that interest at the start of the session.

E. Summary

Each revision corresponds to one of the three framework types; the dashboard, the system narration, and the model's working hypothesis remained synchronized throughout because every superseded response was followed by a fresh `conversation.item.create` of dashboard context before the next `response.create`. We argue this synchrony is the practical value of treating barge-in as a structural supersession signal rather than merely as audio cancellation.

VI. User Study and Technical Benchmark

A. Design

The study is designed to test the paper's central claim rather than general usability: whether a full-duplex supersession pathway makes analytical intent revision in EDA faster, more natural, and more controllable. The primary causal comparison is **not** between a realtime speech model and a cascaded speech pipeline. Instead, we compare two conditions that share the same model, voice, prompt, tool schema, dashboard, backend, VAD configuration, and context-reinjection logic. The only intended difference is whether user speech during assistant output triggers immediate response cancellation, epoch invalidation, and stale-action discard.

We plan a within-subjects, counterbalanced study with N=12–16 participants. Participants must have basic data-analysis experience, but need not know the Olist domain in advance. Each participant completes two open-ended exploration tasks on the Olist dataset, one per primary condition; task–condition assignment and condition order are counterbalanced. Each task lasts 12–15 minutes after a short tutorial.

- **Condition A (VerbalVis-FullDuplex):** audio is handled by the `gpt-realtime-2` session; the user can speak during assistant output; barge-in cancels or truncates the current response, increments `turn_epoch`, invalidates stale tool work, and triggers replanning against reinjected dashboard context.
- **Condition B (VerbalVis-NoBarge):** the system uses the same `gpt-realtime-2` session, voice, prompt, VAD settings, dashboard, backend, tool schema, and context-injection logic. However, user speech during assistant output is queued until the current response completes; it does not trigger `response.cancel`, `conversation.item.truncate`, immediate epoch

invalidation, or stale-tool discard. This preserves the realtime model family while removing the supersession policy.

Supplementary ecological baseline. We additionally define **VerbalVis-Cascade** as an ecological baseline rather than the main causal comparison. In this condition, user speech is transcribed by ASR, passed to a text LLM using the same dashboard tool schema and Olist backend, and synthesized back to speech with TTS. The user must wait for the system to finish before the next request is processed. This baseline represents the speech-wrapped, turn-based text-LLM VA regime implicit in systems such as LightVA and DashChat, but it necessarily changes model type, speech pipeline, ASR/TTS latency, audio quality, turn-taking policy, and response timing. We therefore report Cascade results as supplementary context, not as evidence that the barge-in mechanism alone caused any observed advantage.

B. Tasks

Tasks are written to invite open-ended EDA while naturally eliciting the three revision types in our framework. Participants are not instructed to interrupt. The tutorial only explains the interaction regime of each condition: in FullDuplex they may speak whenever they want to redirect the analysis; in NoBarge their speech during assistant output is accepted but processed after the current response finishes. If Cascade is run, participants are told that it is a turn-based speech interface whose next request is processed only after the current response completes.

- **Customer satisfaction task.** "Explore what factors may be associated with low customer review scores. Produce three findings and one follow-up recommendation."
- **Sales, logistics, and region task.** "Investigate unusual order or revenue patterns and whether they relate to delivery or regional differences. Produce three findings and one follow-up recommendation."

C. Measurements

Quantitative.

- *Intent revision count* — number of user-initiated direction changes per session, coded from session video, transcript, dashboard events, and the `barge_in.log` timeline.
- *Revision type distribution* — proportion of revisions coded as Goal Shift, Hypothesis Correction, or Scope Narrowing.
- *Exploration breadth* — number of distinct analytical dimensions touched (fields, views, filter expressions) during the session.
- *Insight count* — number of substantive findings reported in the post-task summary, coded by two raters with Cohen's κ reported.
- *Time-to-revision* — elapsed time between the start of the redirecting utterance and the first system action aligned with the new intent, measured from video and event logs.
- *Wasted output time* — time during which the system continues producing the old analytical direction after the participant has begun redirecting.
- *Queue delay* — in NoBarge, the interval between redirecting speech onset and the moment the queued request begins processing.
- *Time-to-first-audio (TTFA)* and *tool-call duration*, recorded automatically by the existing `_response_metrics` instrumentation in `realtime.py`; if Cascade is run, ASR, text-LLM, and TTS latencies are logged separately and reported as component-level ecological costs.

Qualitative.

- Think-aloud transcripts, coded thematically.
- Semi-structured post-task interviews (10–15 min).
- NASA-TLX (six standard sub-scales).
- Perceived Control scale (3 Likert-7 items, e.g., "I felt able to redirect the analysis whenever I wanted.").

D. Episode Coding

The unit of analysis is a *revision episode*. Two coders inspect synchronized transcripts, screen recordings, dashboard event logs, and audio timelines. For each candidate episode, coders determine (i) whether it is a valid analytical intent revision rather than a clarification, backchannel, or unrelated speech fragment; (ii) whether it is Goal Shift, Hypothesis Correction, or Scope Narrowing; (iii) whether the system successfully responded to the revised intent; and (iv) the time from the start of the redirecting utterance to the first aligned system action, such as a tool call, dashboard update, or explicit spoken acknowledgment. Disagreements are resolved through discussion after reporting inter-rater agreement.

E. Hypotheses

- **H1:** Intent revision count is higher under VerbalVis-FullDuplex than under VerbalVis-NoBarge.
- **H2:** VerbalVis-FullDuplex reduces time-to-revision, queue delay, and wasted output time relative to VerbalVis-NoBarge.
- **H3:** VerbalVis-FullDuplex increases exploration breadth and insight count relative to VerbalVis-NoBarge.
- **H4:** Perceived control is higher under VerbalVis-FullDuplex, while NASA-TLX is not significantly higher.
- **H5:** The advantage of VerbalVis-FullDuplex is largest for *Hypothesis Correction* and *Scope Narrowing*, because these revisions are most costly when users must wait through an explanation they already intend to reject or constrain.

F. Technical Benchmark

To separate mechanism-level behavior from participant variability, we additionally plan a 250-trial automated benchmark comparing VerbalVis-FullDuplex and VerbalVis-NoBarge. Scripted audio scenarios cover five interruption families: no interruption, early goal shift, late goal shift, hypothesis correction, and scope narrowing. Each scenario is repeated across multiple seeds and dashboard states. The benchmark reports speech-onset detection latency, cancel/truncation latency, stale-tool discard rate, queued-request delay, post-supersession tool

correctness, dashboard-context consistency, and component-level latency. The user study tests whether supersession-enabled full-duplex interaction changes analytical behavior and perceived control; the technical benchmark tests whether the mechanism itself reliably cancels stale work, discards stale calls, and preserves dashboard-context consistency.

G. Results (to be reported)

Placeholder. Tables and figures will report per-condition means and within-subject differences for the primary FullDuplex–NoBarge comparison with Wilcoxon signed-rank tests and effect sizes. Cascade, if collected, will be reported in a separate supplementary table to contextualize ecological latency and user experience, not to support the causal barge-in claim. Qualitative themes will be illustrated with representative quotations. The instrumentation already in place yields TTFA, per-turn latency, tool-call duration, stale-call status, and a per-session dashboard context snapshot for every tool invocation, providing both quantitative and trace data.

H. Anticipated Discussion

We expect the analyses to show that (a) the three revision types in our framework appear in the data with distinguishable distributions across participants; (b) the FullDuplex advantage over NoBarge is largest on Hypothesis Correction and Scope Narrowing episodes, where delayed processing forces users to passively witness explanations they already intend to reject or constrain; and (c) Perceived Control correlates positively with intent revision count and negatively with wasted output time. We further expect the automated benchmark to show that the gain is not merely due to the realtime model family, but to the supersession pathway that cancels stale output and preserves dashboard-context consistency.

VII. Discussion

Limitations. The current prototype implements five of the fifteen taxonomy operations as schema-based tools; while these operations cover the running example and planned study tasks, claims about the taxonomy's completeness require broader deployment. VerbalVis treats all barge-in events with the same downstream processing and does not classify revision type at runtime; the three types in Section III are analytical categories, not separate system policies. The planned study size (N=12–16) is appropriate for an exploratory within-subjects system study but still limits statistical power for subgroup analysis. VerbalVis-NoBarge is a controlled ablation rather than a naturally deployed product: it preserves the realtime model family while disabling immediate supersession, which is useful for causal attribution but less ecologically familiar. Conversely, VerbalVis-Cascade is ecologically meaningful because it represents a speech-wrapped text-LLM VA regime, but it changes model type, speech pipeline, latency decomposition, audio quality, and turn-taking policy; we therefore treat it only as supplementary context. Finally, the Olist dataset, while rich, is domain-specific; generalization to clinical, financial, or scientific datasets remains future work.

Implications. Treating barge-in as a structural supersession signal expands the conversational VA design space without requiring the system to infer the user's meaning from timing alone. The interruption event gives the controller permission to protect the analytical state from stale work; the redirecting utterance provides the content of the revised intent. Future systems may also use the *timing* of an interruption (early vs. late in a response) as a weak cue about whether the user rejects the direction, the evidence, or the conclusion, but such use should remain subordinate to the utterance content and dashboard context.

Generalization beyond voice. The framework is voice-motivated but not voice-bound. Any modality with a true interruption primitive — including streaming-LLM text chat with mid-response edits, or gestural systems that can preempt animations — can be analyzed through the same Intent Evolution lens. We see VerbalVis as a concrete instantiation of a broader design pattern.

Future work. Beyond completing the tool taxonomy and broadening data domains, we plan (i) type-aware interruption handling that differentiates Goal Shift, Hypothesis Correction, and Scope Narrowing at runtime, (ii) a longitudinal study examining whether users' intent-revision strategies stabilize over repeated sessions, and (iii) an extension to multi-analyst sessions where barge-ins from different participants might conflict or require negotiated replanning.

VIII. Conclusion

We have presented VerbalVis, a full-duplex conversational visual analytics system motivated by the observation that user barge-in can serve as a structural supersession signal during analytical work. The Intent Evolution framework identifies three recurring revision types — Goal Shift, Hypothesis Correction, and Scope Narrowing — and motivates a reusable supersession control protocol for conversational EDA. Built on `gpt-realtime-2`, response epochs, schema-based tool calling, and an in-memory DuckDB analytics layer with continuously reinjected dashboard context, VerbalVis demonstrates how full-duplex speech can move intent revision from a between-turn afterthought to a continuously available action. A case study on the Olist e-commerce dataset illustrates all three revision types in a single session; a planned within-subjects user study will make its primary comparison between VerbalVis-FullDuplex and VerbalVis-NoBarge, with VerbalVis-Cascade retained only as a supplementary ecological baseline; and a 250-trial technical benchmark will test the reliability of the supersession mechanism. Together, these evaluations separate the human effect of full-duplex redirection from the mechanism-level question of whether stale work can be canceled or discarded while dashboard context remains consistent.

References

[1] P. Pirolli and S. Card, "The sensemaking process and leverage points for analyst technology as identified through cognitive task analysis," in *Proceedings of the International Conference on Intelligence Analysis*, vol. 5. McLean, VA, USA, 2005, pp. 2–4.

[2] S. Kandel, A. Paepcke, J. M. Hellerstein, and J. Heer, "Enterprise data analysis and visualization: An interview study," *IEEE Transactions on Visualization and Computer Graphics*, vol. 18, no. 12, pp. 2917–2926, 2012.

[3] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2015, pp. 489–500.

[4] V. Setlur, S. E. Battersby, M. Tory, R. Gossweiler, and A. X. Chang, "Eviza: A natural language interface for visual analysis," in *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2016, pp. 365–377.

[5] A. Narechania, A. Srinivasan, and J. T. Stasko, "NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 369–379, 2021.

[6] C. Wang, B. Lee, S. M. Drucker, D. Marshall, and J. Gao, "Data formulator 2: Iteratively creating rich visualizations with ai," 2024.

[7] Y. Zhao, J. Wang, L. Xiang, X. Zhang, Z. Guo, C. Turkay, Y. Zhang, and S. Chen, "LightVA: Lightweight visual analytics with LLM agent-based task planning and execution," *IEEE Transactions on Visualization and Computer Graphics*, 2024, early Access.

[8] S. Shen, Z. Lin, W. Liu, C. Xin, W. Dai, S. Chen, X. Wen, and X. Lan, "Dashchat: Interactive authoring of performance dashboard design prototypes through conversation with llm-powered agents," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2026.

[9] OpenAI, "Realtime API documentation," https://platform.openai.com/docs/guides/realtime, 2026, accessed: 2026-06-25.

[10] A. Srinivasan and J. T. Stasko, "Orko: Facilitating multimodal interaction for visual exploration and analysis of networks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 24, no. 1, pp. 511–521, 2018.

[11] R. Mitra, A. Narechania, A. Endert, and J. T. Stasko, "Facilitating conversational interaction in natural language interfaces for visualization," in *2022 IEEE Visualization and Visual Analytics Conference*. IEEE, 2022, pp. 6–10.

[12] A. Srinivasan and V. Setlur, "Snowy: Recommending utterances for conversational visual analysis," in *Proceedings of the 34th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2021, pp. 864–880.

[13] V. Dibia, "LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 113–126.

[14] Y. Zhao, X. Shu, L. Fan, L. Gao, Y. Zhang, and S. Chen, "Proactiveva: Proactive visual analytics with llm-based ui agent," *arXiv preprint arXiv:2507.18165*, 2025, preprint.

[15] L. Xie, C. Zheng, H. Xia, H. Qu, and Z.-T. Chen, "WaitGPT: Monitoring and steering conversational LLM agent in data analysis with on-the-fly code visualization," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2024.

[16] T.-E. Lin, Y. Wu, F. Huang, L. Si, J. Sun, and Y. Li, "Duplex conversation: Towards human-like interaction in spoken dialogue systems," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2022, pp. 3299–3308.

[17] P. Wang, S. Lu, Y. Tang, S. Yan, W. Xia, and Y. Xiong, "A full-duplex speech dialogue scheme based on large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 13 372–13 403.

[18] B. Veluri, B. N. Peloquin, B. Yu, H. Gong, and S. Gollakota, "Beyond turn-based interfaces: Synchronous LLMs as full-duplex dialogue agents," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 21 390–21 402.

[19] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, "Moshi: A speech-text foundation model for real-time dialogue," 2024.

[20] H. Zhang, J. Chen, D. Wu, Y. Li, Y. Zhang, X. T. Zhang, C. Liu, Q. Lin, Y. Peng, H. Liu, E. S. Chng, C. Yan, B. Wu, Y. Huang, X. Yang, and F. Tian, "Duplexsla: A full-duplex spoken language model with synchronized speech, language, and action," *arXiv preprint arXiv:2605.20755*, 2026, preprint.

[21] G.-T. Lin, C. Chen, Z. Chen, and H.-y. Lee, "Full-Duplex-Bench-v3: Benchmarking tool use for full-duplex voice agents under real-world disfluency," 2026.

[22] D. M. Russell, M. J. Stefik, P. Pirolli, and S. K. Card, "The cost structure of sensemaking," in *Proceedings of the INTERCHI '93 Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1993, pp. 269–276.

[23] D. Sacha, A. Stoffel, F. Stoffel, B. C. Kwon, G. Ellis, and D. A. Keim, "Knowledge generation model for visual analytics," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, no. 12, pp. 1604–1613, 2014.

[24] E. D. Ragan, A. Endert, J. Sanyal, and J. Chen, "Characterizing provenance in visualization and data analysis: An organizational framework of provenance types and purposes," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 1, pp. 31–40, 2016.

[25] P. H. Nguyen, K. Xu, A. Wheat, B. L. W. Wong, S. Attfield, and B. Fields, "SensePath: Understanding the sensemaking process through analytic provenance," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 1, pp. 41–50, 2016.

[26] J. Heer, J. Mackinlay, C. Stolte, and M. Agrawala, "Graphical histories for visualization: Supporting analysis, communication, and evaluation," *IEEE Transactions on Visualization and Computer Graphics*, vol. 14, no. 6, pp. 1189–1196, 2008.

[27] K. Davidson, L. Lisle, K. Whitley, D. A. Bowman, and C. North, "Exploring the evolution of sensemaking strategies in immersive space to think," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 12, pp. 5294–5307, 2023.

[28] M. Brehmer and T. Munzner, "A multi-level typology of abstract visualization tasks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 19, no. 12, pp. 2376–2385, 2013.

[29] J. S. Yi, Y.-a. Kang, J. Stasko, and J. A. Jacko, "Toward a deeper understanding of the role of interaction in information visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 13, no. 6, pp. 1224–1231, 2007.

[30] OpenAI, "Realtime models prompting guide," https://platform.openai.com/docs/guides/realtime-models-prompting, 2026, accessed: 2026-06-25.